Seurat_tutorial

Meryem Stroosma

2025-12-01

Content

Introduction

This RMarkdown contains a brief summary of the R package Seurat. Seurat is used for the analysis of single-cell RNA sequencing data, providing tools for data preprocessing, quality control, normalization, dimensionality reduction, clustering, differential expression, and cell-type annotation. Besides analyzing a dataset Seurat can also be used to create a variaty of plots to visualize the data. The summary in this RMarkdown is based on the Seurat guided clustering tutorial.

Purpose

The purpose of this RMarkdown is to understand the different functionalities of Seurat, and to learn how to use Seurat for analyzing the GSE138852 count matrix.

Data

The data that is used in this RMarkdown can be accessed here. This is a dataset of Peripheral Blood Mononuclear Cells (PBMC) freely available from 10X Genomics. This dataset is also used in the Seurat guided clustering tutorial.

Packages

Prior to analysis several packages need to be loaded. The required packages are the following:

- `dplyr`: this package is used by Seurat for data manipulation.
- `Seurat`: this package needs to be loaded prior to running any Seurat specific functions.
- `patchwork`: the patchwork package can be used to combine several plots into a single figure.
- `here`: this package is used to make sure the file paths are reproducible.

Loading the dataset and creating a Seurat object

In order to perform analysis with Seurat, the data first needs to be loaded. This is done with `Read10X()` which is a Seurat function that can be used to load 10X genomics files such as matrices and .tsv files. After loading the dataset, the function `CreateSeuratObject()` is used to create a Seurat object that contains the raw data. In this object several filters can be applied or altered, which is shown in the following sections.

Pre-processing of the dataset

To make sure the data that will be analyzed is reliable it is important that low quality cells are removed from the dataset. In this case the quality doesn't refer to a Phred score (read quality), but to the amount of unique features per cell, the percentage of mitochondrial genes and the total number of molecules per cell;

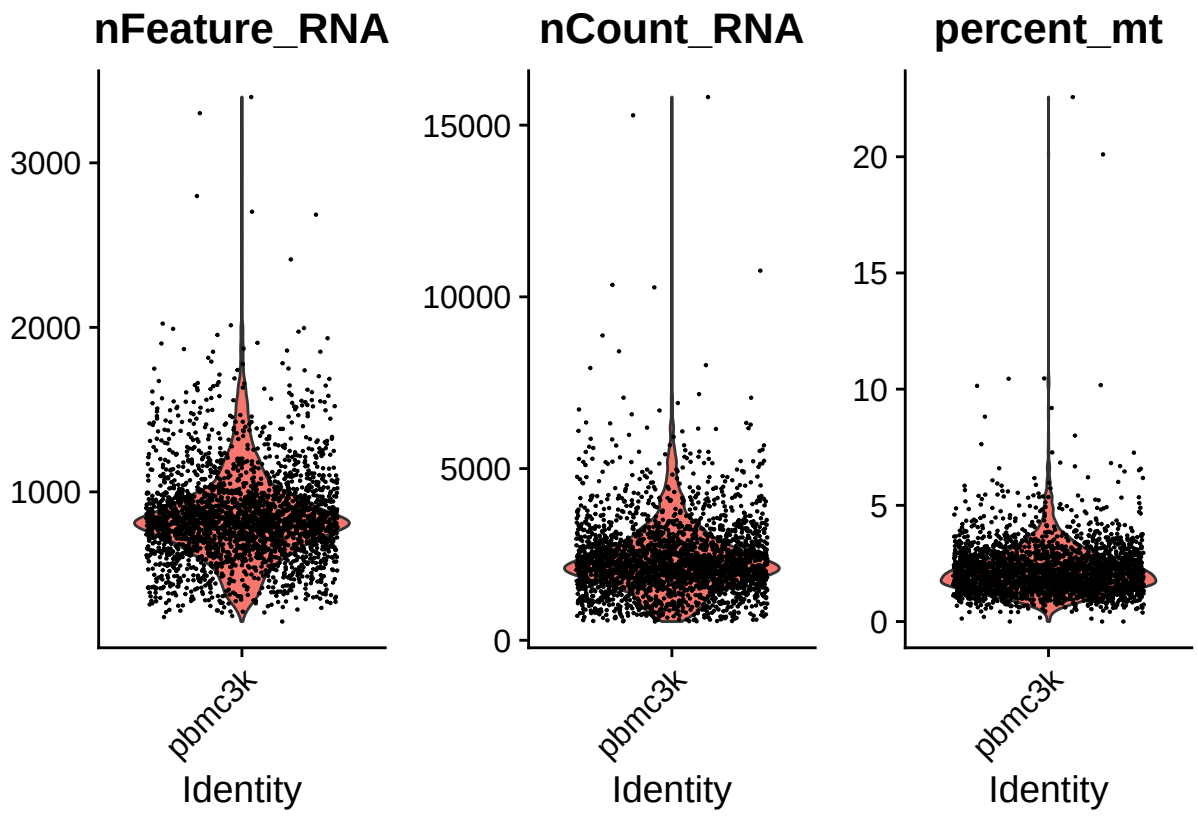
- The number of unique features per cell are relevant since too few features compared to the rest of the cells could mean that sequencing of this particular cell was insufficient or that the droplet did not contain a cell. If the number of features is higher it could mean that two or more cells were captured in the same droplet. In most cases a feature is a gene, but this doesn't always have to be the case, sometimes this might be regulatory RNA instead.
- The number of counts says something about the number of different molecules or Unique Molecular Identifiers (UMIs) in one cell. If this number is too low it could mean that this is a poor capture and if this number is too high it could mean that there might be more then one cell in a droplet.
- The percentage of mitochondrial genes are a good indication about the general health of the cell, if this percentage is high it usually means that the cell is dying or stressed. Since it might not be representative for the condition that is analysed it is better to filter this out.

The first step in pre-processing is adding a column to the dataframe with the percentage of mitochondrial genes. To give a general idea of what the dataframe looks like after adding this column the first five rows are shown in the chunk below. In the first column the cell barcodes are shown, the second column contains the original identity of the sample the third, fourth and fifth column contain the number of the number of counts, unique genes and the percentage of mitochondrial genes.

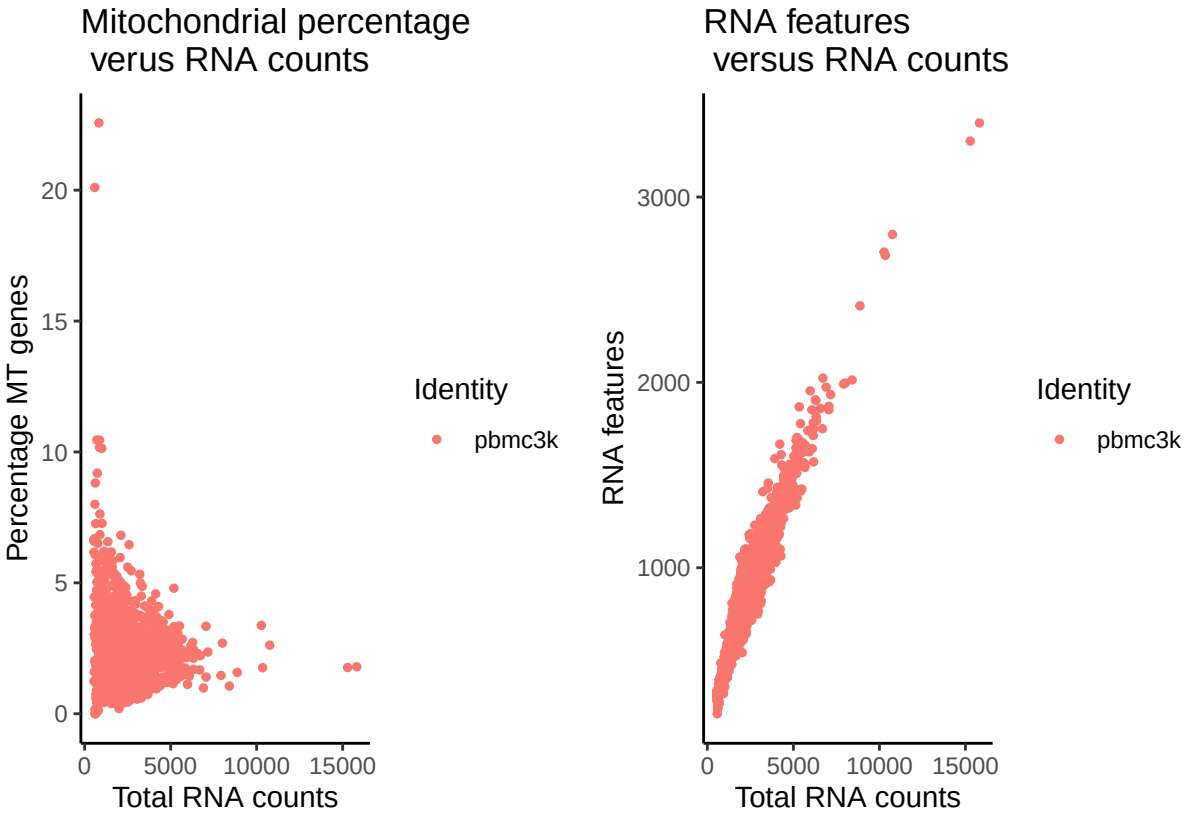
```
##                  orig.ident nCount_RNA nFeature_RNA percent_mt
## AAACATACAACCAC-1     pbmc3k       2419          779  3.0177759
## AAACATTGAGCTAC-1     pbmc3k       4903         1352  3.7935958
## AAACATTGATCAGC-1     pbmc3k       3147         1129  0.8897363
## AAACCGTGCTTCCG-1     pbmc3k       2639          960  1.7430845
## AAACCGTGTATGCG-1     pbmc3k        980          521  1.2244898
```

The dataframe can also be visualised in a plot, in figure 1 a violin plot is shown of the following metrics;

- **nFeature_RNA**: this plot shows the unique number of genes
- **nCount_RNA**: this plot shows the total amount of RNA molecules detected
- **percent_mt**: this plot shows the percentage of mitochondrial genes In the violin plots in figure 1 each dot represents the datapoint for one single cell, on the x-axis the identity is shown, on the y-axis the amount is shown of the metric that is plotted.



With the function `FeatureScatter()` a scatterplot is generated of two QC metrics. The first argument should contain the Seurat object where the QC metrics are stored. Then the x-axis should be defined with the argument `feature1 =`, in figure 2 this was defined as nCount_RNA. The y-axis can be defined in a similar way using `feature2 =`, in the left plot in figure 2 this was defined as *percent_mt* and in the right plot this was defined as *nFeature_RNA*. The title of the plot is the correlation value, this could be changed by the argument plot.cor, which can be set to TRUE or FALSE. `FeatureScatter()` also has several arguments that could be used to modify the plots, for example if the dataset contains cells with different identities these could be grouped in different colours with argument group.by.


Mitochondrial percentage
verus RNA counts
Percentage MT genes
20
15
10
5
0
0
5000
10000
15000
Total RNA counts
Identity
pbmc3k
RNA features
versus RNA counts
RNA features
3000
2000
1000
0
5000
10000
15000
Total RNA counts
Identity
pbmc3k